Un equipo de una persona

Flujos de trabajo con IA agéntica en humanidades digitales

Guillermo Marco Remón
Grupo NLP & IR, ETSI Informática, UNED
Seminario de investigación del grupo INHDIE · 15 de octubre de 2026

Esquema

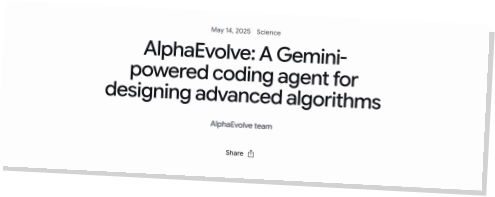
1. Introducción al trabajo con agentes
2. Limitaciones de los agentes
3. Buenas prácticas derivadas de las limitaciones
4. Caso práctico: ¿quién escribió el *Lazarillo*? Con Claude Code

Prompts: `https://claude.ai/artifact/8E14gReCvbXappyn2BPWnf`

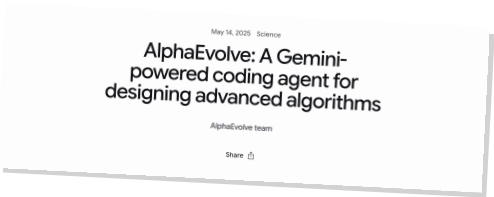
Materiales: github.com/grmarco/taller-agentes-hd

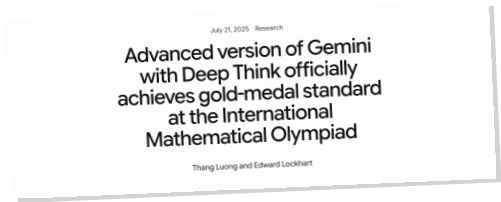
Introducción al trabajo con agentes

Resultados recientes atribuidos a sistemas agénticos



Resultados recientes atribuidos a sistemas agénticos





Resultados recientes atribuidos a sistemas agénticos

May 14, 2025 Science

AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms

AlphaEvolve team

Share



IMAGE CREDITS: JAKUB PORZYCKI/NURPHOTO / GETTY IMAGES

Anthony Ha

OpenAI's 'embarrassing' math

"Hoisted by their own GPTards."

July 21, 2025 Research

Advanced version of Gemini with Deep Think officially achieves gold-medal standard at the International Mathematical Olympiad

Thang Luong and Edward Lockhart

Resultados recientes atribuidos a sistemas agénticos

May 14, 2025 Science

AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms

AlphaEvolve team

Share



IMAGE CREDITS: JAKUB PORZYCKI/NURPHOTO / GETTY IMAGES

Anthony Ha

OpenAI's 'embarrassing' math

"Hoisted by their own GPTards."

July 21, 2025 Research

Advanced version of Gemini with Deep Think officially achieves gold-medal standard at the International Mathematical Olympiad

Thang Luong and Edward Lockhart

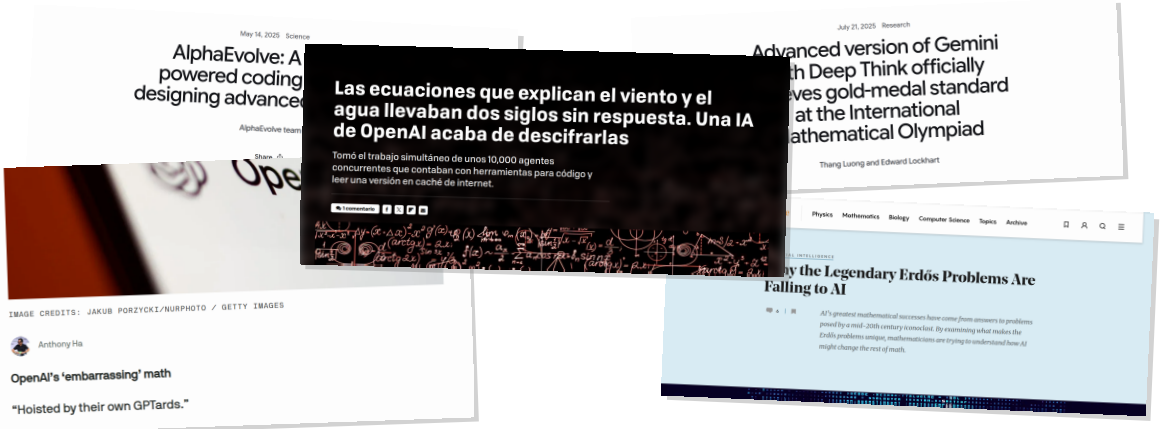
Quanta magazine Physics Mathematics Biology Computer Science Topics Archive

ARTIFICIAL INTELLIGENCE

Why the Legendary Erdős Problems Are Falling to AI

AI's greatest mathematical successes have come from answers to problems posed by a mid-20th century iconoclast. By examining what makes the Erdős problems unique, mathematicians are trying to understand how AI might change the rest of math.

Resultados recientes atribuidos a sistemas agénticos



Resultados recientes atribuidos a sistemas agénticos



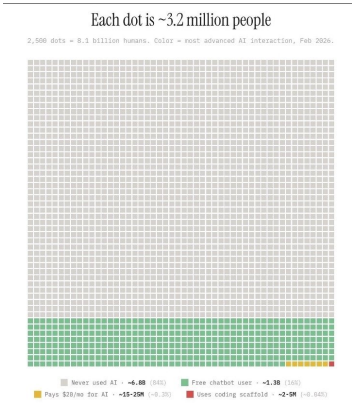
Alcance real de esos resultados

Resultado	Qué se afirma	Matiz
Navier–Stokes (sep. 2026)	OpenAI: demostración del caso *forzado* (165 pp., formalizada en Lean; ~10 000 agentes, 88 h)	No es el problema del milenio: el Clay lo mantiene abierto; sin verificación independiente; disputa de prioridad
Erdős (oct. 2025)	«GPT-5 resuelve 10 problemas abiertos»	Eran resultados ya publicados: búsqueda bibliográfica, no demostraciones. Se retiró
Erdős (2026)	Problema #728; refutación de la conjetura de distancias unitarias (mayo)	Resultados genuinos según los matemáticos; margen mínimo, mejorado por humanos en semanas
Olimpiada Matemática (jul. 2025)	Oro (35/42) de Google DeepMind y de OpenAI	Solo el de DeepMind lo corrigieron los coordinadores de la IMO
Biología, Anthropic (sep. 2026)	~950 agentes Claude localizan una familia de retrotranscriptasas con estructura tipo CRISPR	Preprint sin revisar; función no demostrada; el trabajo de laboratorio lo hicieron personas

Fuentes: Google DeepMind (14-5-2025, 21-7-2025); TechCrunch (19-10-2025, 23-9-2026); Quanta Magazine (3-8-2026); Xataka (8-9-2026); The Scientist; Clay Mathematics Institute. Detalle en `notas/logros.md`.

Grado de adopción

Interacción más avanzada con IA de cada persona, febrero de 2026 (estimación)



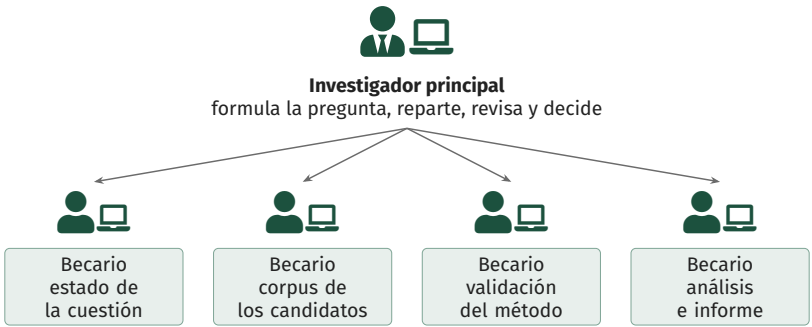
- 84 %: nunca ha usado IA.
- 16 %: chat gratuito.
- 0,3 %: paga una suscripción.
- **0,04 %**: trabaja con un agente que programa.

El taller trata del último grupo.

Damian Player (@damianplayer), X, 21-2-2026. Cifras aproximadas, sin fuente de datos declarada; el 84 % coincide con estimaciones de Microsoft para 2025.

Organización de un proyecto de humanidades digitales sin agentes

Ejemplo: la autoría del *Lazarillo*



plazo: semanas o meses · coordinación: reuniones y correo · coste: contratos

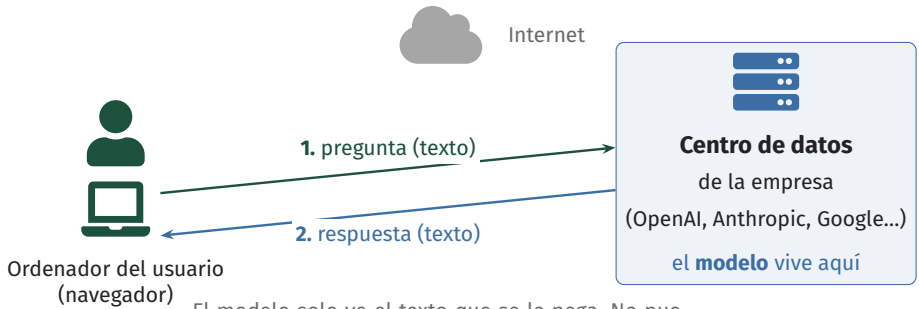
Organización del mismo proyecto con agentes

El investigador principal encarga, aprueba el plan y revisa; los agentes ejecutan



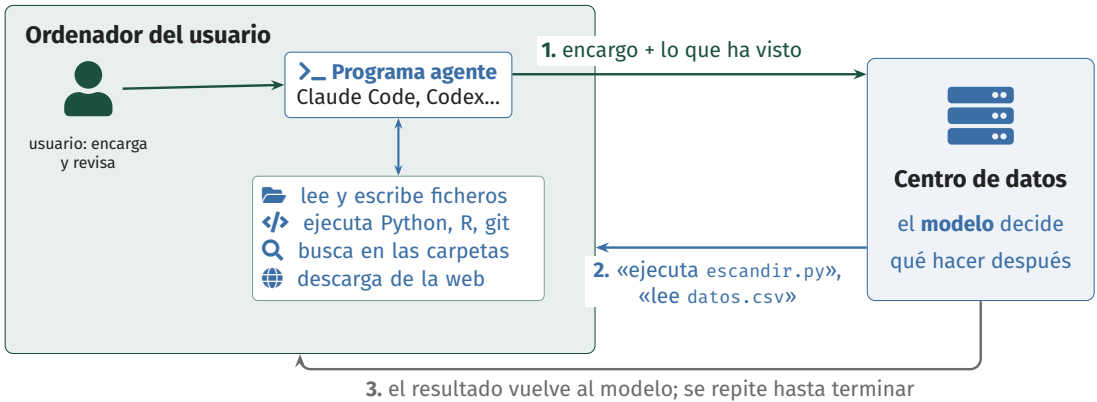
plazo: horas · coordinación: instrucciones escritas · coste: tokens

Arquitectura de un chat



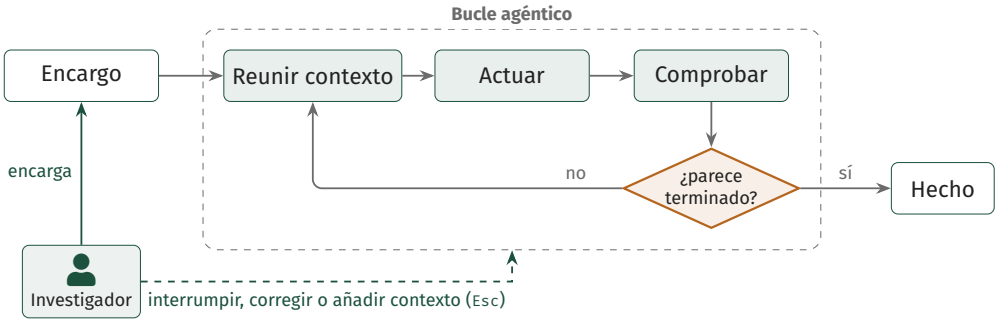
El modelo solo ve el texto que se le pega. No puede abrir ficheros ni ejecutar nada en el ordenador: **el usuario** copia, pega, ejecuta y le cuenta qué ha fallado.

Arquitectura de un agente



El modelo sigue en el centro de datos. La diferencia es un programa en el ordenador del usuario que ejecuta lo que el modelo pide y le devuelve el resultado.

El bucle agéntico



El agente decide por sí mismo cuándo ha terminado: para cuando el trabajo *parece* hecho.

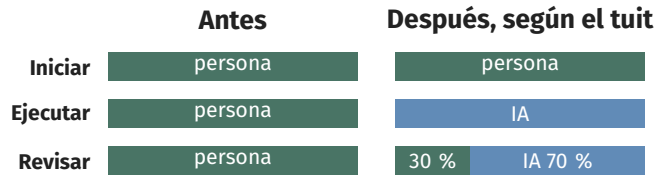
Adaptado de *How Claude Code works*, code.claude.com/docs.

Distribución de funciones: iniciar, ejecutar, revisar

> *«La única cosa que aún nos hace relevantes es nuestro papel como iniciadores. El papel del ejecutor ya está muerto. Y el del revisor, muerto en un 70 %.»*

Usuario de X, 2026 (traducción).

https://x.com/Gegam245074/status/2108294261359706318



Las cifras son la impresión de un usuario, no una medición.

Una analogía: la mecanización del campo



Siega a mano



Una persona y una máquina

En este taller vamos a aprender a llevar el tractor.

Cómo llevar el tractor

1. Instala Claude Code, Codex u OpenCode.
2. Pídele lo que quieras.

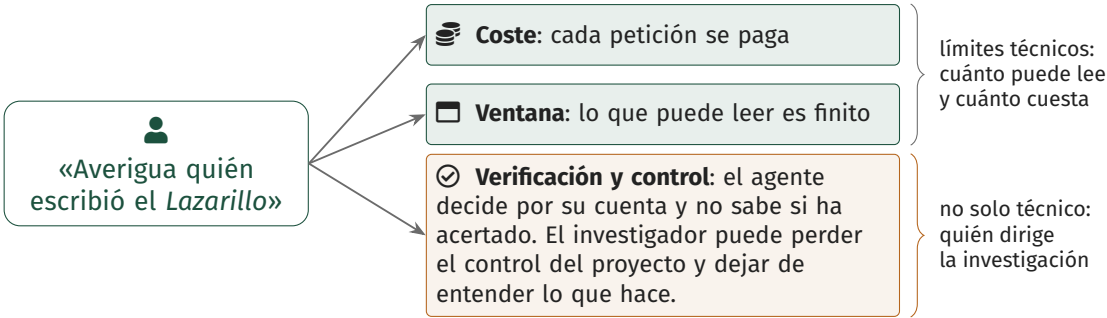
Cómo llevar el tractor

1. Instala Claude Code, Codex u OpenCode.
2. Pídele lo que quieras.

Muchas gracias. ¿Preguntas?

Limitaciones de los agentes

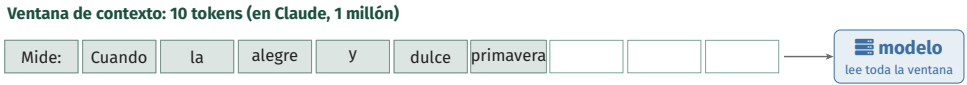
Por qué no basta con pedírselo



Cada petición cuesta y lo que el modelo puede leer es finito. Pero el límite más serio es otro: si el agente toma decisiones por su cuenta, el investigador puede dejar de entender su propio proyecto.

Tokens, ventana de contexto y generación autorregresiva

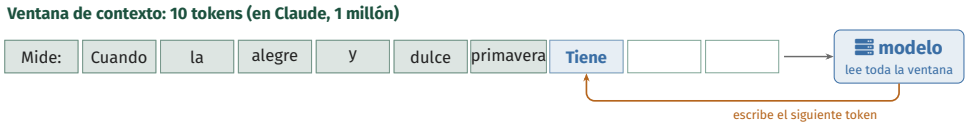
Un solo modelo lee toda la ventana y escribe un token cada vez



En verde, la petición; en azul, lo que escribe el modelo. Un token suele ser una palabra corta o un fragmento de palabra; aquí se simplifica a palabras.

Tokens, ventana de contexto y generación autorregresiva

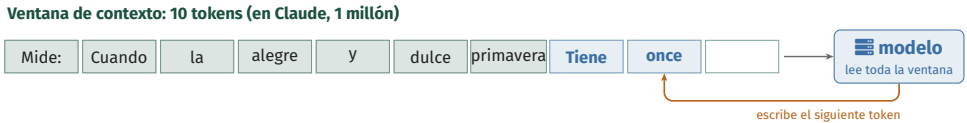
Un solo modelo lee toda la ventana y escribe un token cada vez



En verde, la petición; en azul, lo que escribe el modelo. Un token suele ser una palabra corta o un fragmento de palabra; aquí se simplifica a palabras.

Tokens, ventana de contexto y generación autorregresiva

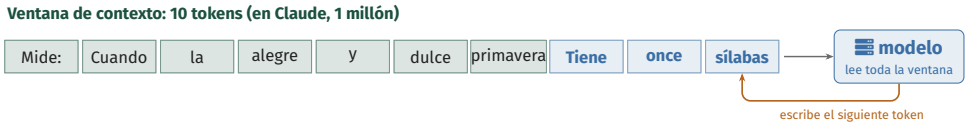
Un solo modelo lee toda la ventana y escribe un token cada vez



En verde, la petición; en azul, lo que escribe el modelo. Un token suele ser una palabra corta o un fragmento de palabra; aquí se simplifica a palabras.

Tokens, ventana de contexto y generación autorregresiva

Un solo modelo lee toda la ventana y escribe un token cada vez



En verde, la petición; en azul, lo que escribe el modelo. Un token suele ser una palabra corta o un fragmento de palabra; aquí se simplifica a palabras.

Tokens, ventana de contexto y generación autorregresiva

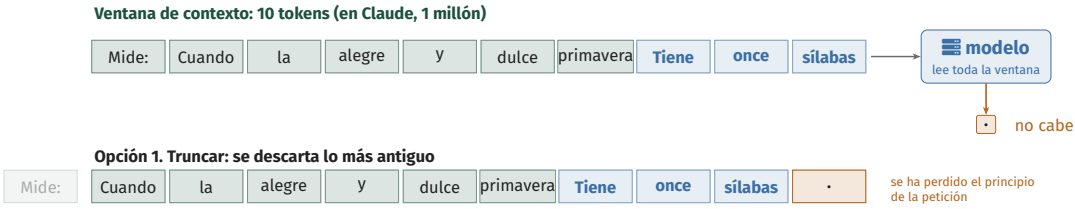
Un solo modelo lee toda la ventana y escribe un token cada vez



En verde, la petición; en azul, lo que escribe el modelo. Un token suele ser una palabra corta o un fragmento de palabra; aquí se simplifica a palabras.

Tokens, ventana de contexto y generación autorregresiva

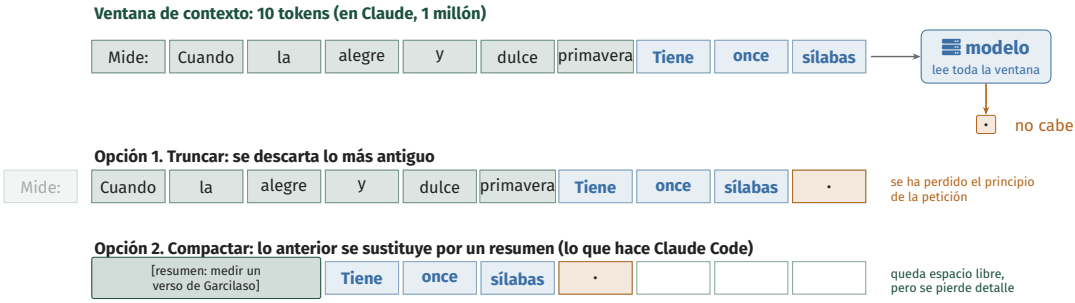
Un solo modelo lee toda la ventana y escribe un token cada vez



En verde, la petición; en azul, lo que escribe el modelo. Un token suele ser una palabra corta o un fragmento de palabra; aquí se simplifica a palabras.

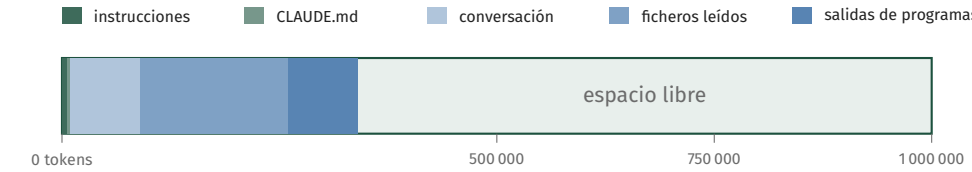
Tokens, ventana de contexto y generación autorregresiva

Un solo modelo lee toda la ventana y escribe un token cada vez



En verde, la petición; en azul, lo que escribe el modelo. Un token suele ser una palabra corta o un fragmento de palabra; aquí se simplifica a palabras.

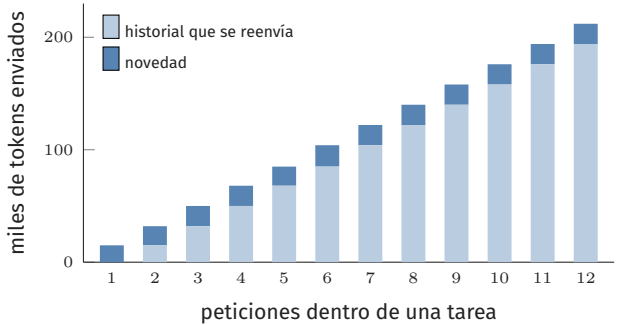
La ventana de contexto: un millón de tokens



Todo ocupa ventana: la conversación, los ficheros leídos y los resultados de los programas.

Esquema de una sesión larga. 1 millón de tokens ≈ 555 000 palabras en inglés; en español, algo menos.

Cada petición vuelve a enviar todo el contexto



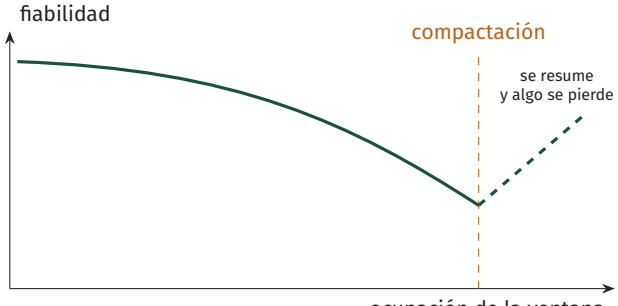
Con suscripción no se pagan dólares, sino cuota de uso.

Cada lectura de un fichero o ejecución de un programa es una petición nueva, y con ella se reenvía todo lo anterior.

Ejemplo con Opus y 200 000 tokens acumulados: 0,80 $ por petición; con la caché, unos 0,04 $.

Claude Code, *Manage costs effectively*: «Claude Code sends your full conversation with every request».

Ocupación de la ventana y rendimiento



esquema cualitativo

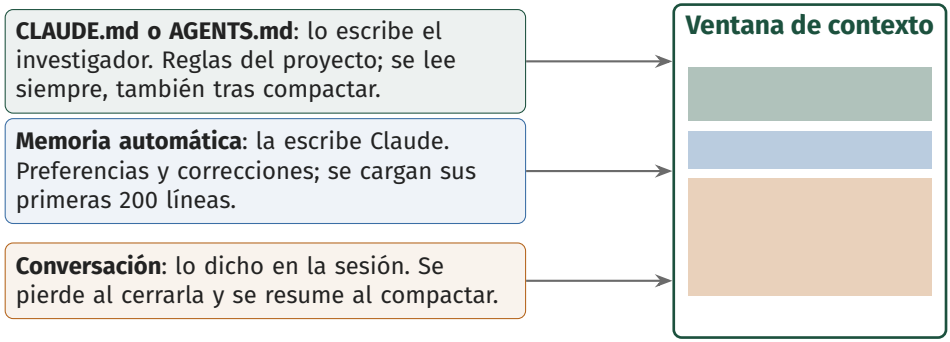
«Performance degrades as context fills.»

Al llenarse la ventana, Claude Code resume la conversación y sigue; en el resumen se pierden detalles, sobre todo las instrucciones del principio.

Anthropic, *Best practices for Claude Code.*

Memoria entre sesiones

Cada sesión empieza con la ventana vacía



Las dos memorias son contexto, no órdenes: el modelo puede no seguirlas. Lo que debe cumplirse siempre se impone con un *hook*.

Claude Code, *How Claude remembers your project*. Codex, Cursor y otros leen `AGENTS.md`.

Qué escribir en CLAUDE.md

Incluir

- Lo que el agente no puede deducir: la pregunta, las fuentes admitidas, las convenciones del proyecto.
- Normas propias: «distinguir afinidad de autoría», «solo dominio público».
- Errores que ya ha cometido dos veces.
- Cómo se comprueba el trabajo.

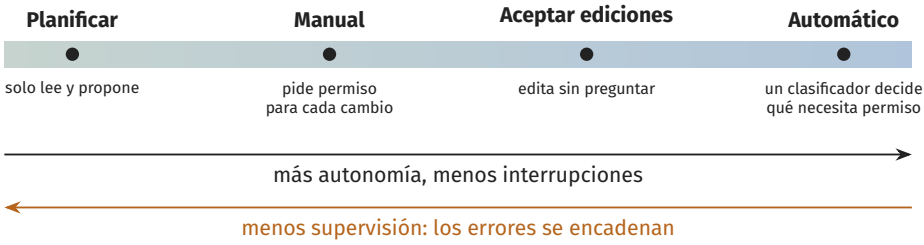
Excluir

- Lo que puede averiguar leyendo los ficheros.
- Explicaciones largas y tutoriales.
- Información que cambia a menudo.
- Obviedades como «trabaja con rigor».

Menos de 200 líneas. Para cada línea: *¿si se borrara, se equivocaría el agente?* Si no, sobra.

Anthropic, *Best practices for Claude Code*: «Would removing this cause Claude to make mistakes? If not, cut it».

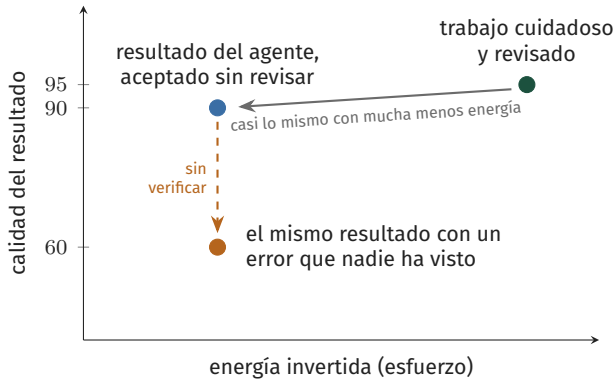
Autonomía y supervisión



Redes de seguridad: deshacer cambios · git · reglas obligatorias (*hooks*)

Modos de permiso de Claude Code (*Permission modes*); Anthropic, *Building effective agents*.

El principio del mínimo esfuerzo



La selección natural favorece las conductas que obtienen más con menos energía.

Si un resultado aceptable cuesta mucha menos energía, el esfuerzo de comprobarlo pierde su incentivo.

La revisión es el trabajo que más fácilmente se abandona.

Zipf, G. K. (1949), *Human Behavior and the Principle of Least Effort*; MacArthur y Pianka (1966), «On optimal use of a patchy environment», *The American Naturalist* 100.

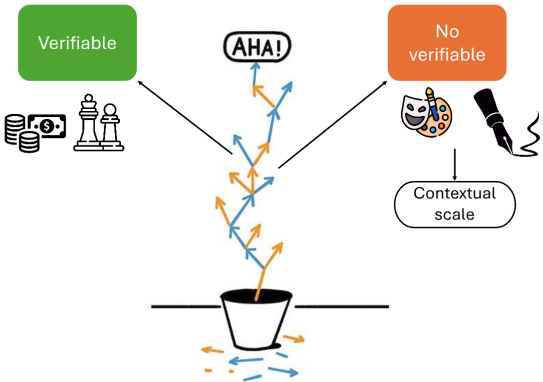
El problema de la verificación



- Conformarse con 90 cuando se podía llegar a 95.
- Entregar 60 sin darse cuenta, por falta de ojo crítico.
- Pedir es fácil; **comprobar** lo que ha hecho el agente es lo difícil, y no se puede delegar del todo.

@biochem_fan, 13-10-2026: https://x.com/biochem_fan/status/2109773144064237844

Tareas verificables y no verificables



Los modelos mejoran más deprisa donde el resultado se puede comprobar automáticamente: una partida se gana o se pierde, un programa compila o no.

Donde el criterio depende del contexto y del juicio experto, como en la escritura o la interpretación, la mejora es más lenta y la comprobación recae en las personas.

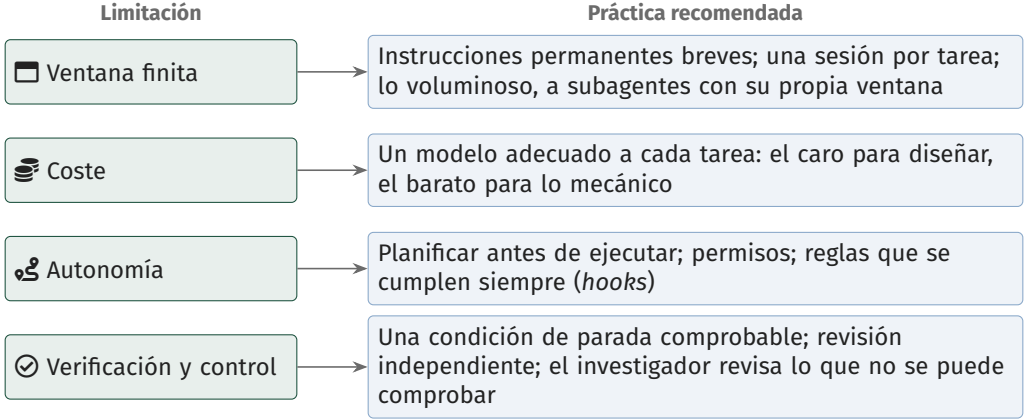
Grados de verificabilidad

	Fuera de las humanidades	En el caso del *Lazarillo*
Verificable hay una comprobación automática	El programa compila y pasa sus pruebas. La demostración se valida con un verificador formal (Lean).	La cita aparece literalmente en la edición. La referencia bibliográfica existe. El texto descargado empieza y termina donde la obra.
Por aproximación hay un criterio externo parcial	La traducción se compara con traducciones de referencia. El diagnóstico, con casos ya confirmados.	El método acierta con textos de autor conocido. La escansión automática coincide con una muestra anotada a mano.
No verificable solo cabe el juicio experto	¿Es buena esta novela? ¿Es convincente este ensayo?	¿Es erasmista la ironía del *Lazarillo*? ¿Qué pesa más, la estadística o el testimonio documental?

Buena parte del trabajo consiste en llevar lo que se pueda a las dos primeras filas.

Buenas prácticas derivadas de las limitaciones

Prácticas recomendadas para cada limitación



Todas se muestran en el caso práctico, en el momento en que hacen falta.

Una condición de parada comprobable

La práctica que más subraya la documentación: «Give Claude a way to verify its work»

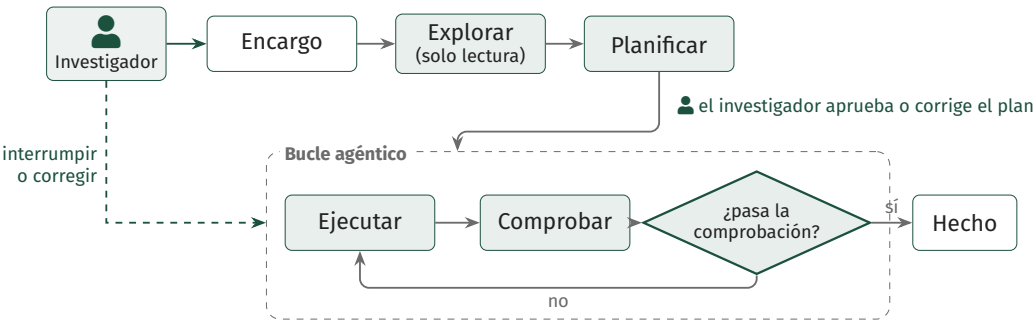
Sin condición

Construye el corpus de los candidatos.

Con condición

Construye el corpus de los candidatos. Para cada texto, comprueba que tiene más de 5 000 palabras, que empieza y termina donde la obra y que está anotado de dónde sale. Termina cuando todos lo cumplan y muéstrame la tabla.

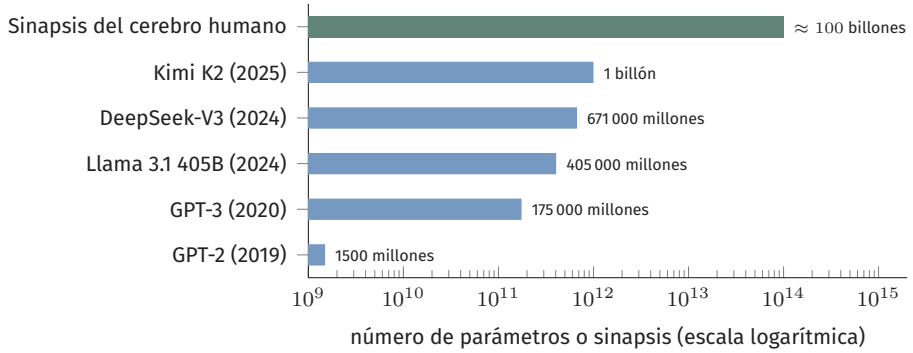
Explorar, planificar, ejecutar y comprobar



Planificar compensa cuando no está claro cómo hacerlo; si la tarea cabe en una frase, se puede pasar directamente a ejecutar.

Anthropic, *Best practices for Claude Code*: «Explore first, then plan, then code».

Tamaño de los modelos: parámetros y sinapsis



Un parámetro se parece más a una sinapsis que a una neurona. Anthropic, OpenAI y Google no publican el tamaño de sus modelos.

Sinapsis: orden de magnitud (Drachman, 2005). Meta (2024), DeepSeek (2024), Moonshot (2025), OpenAI (2019, 2020).

Modelos de Anthropic: papel y precio

Modelo	Papel	Ejemplo de encargo	Precio*
Fable 5.1	Problemas realmente difíciles	*«Revisa esta demostración y localiza el paso que no se sostiene»*	10 / 50
Opus 5.5	Diseño y planificación	*«Diseña cómo validar el método y reparte el trabajo en subagentes»*	4 / 20
Sonnet 5.5	Ejecución	*«Busca los estudios sobre la autoría del Lazarillo y resume método y conclusión»*	2 / 10
Haiku 5.5	Ejecución barata de tareas muy acotadas	*«Descarga y limpia este texto y cuenta sus palabras» (repetido con cada obra)*	0,10 / 0,50

*Dólares por millón de tokens de entrada / salida. Los cuatro tienen una ventana de 1 millón de tokens. Opus cuesta 40 veces más que Haiku.

Precios de la API de Anthropic, consultados el 14-10-2026 (platform.claude.com/docs). Haiku: peticiones de hasta 100 000 tokens.

Caso práctico: ¿quién escribió el *Lazarillo*?

Planteamiento del problema



Burgos, Juan de Junta, 1554

- Anónimo de 1554. Candidatos: Alfonso y Juan de Valdés, Hurtado de Mendoza, Horozco, Cervantes de Salazar, Arce de Otálora…
- Al menos ocho estudios estilométricos (2016–2024) que **no coinciden**: Otálora, Juan de Valdés; Alfonso de Valdés, descartado.
- Textos en dominio público: el *Lazarillo* (unas 18 000 palabras) y obras de los Valdés y de Hurtado. Faltan otros, Otálora entre ellos.

Objetivo del taller: reproducir el problema con agentes y valorar críticamente qué se puede afirmar.

Paso 0: preparación del entorno

Herramienta	Función	Material de iniciación
Git	Guardar versiones y deshacer lo que haga el agente	Software Carpentry, *El control de versiones con Git*: swcarpentry.github.io/git-novice-es
Miniforge	Python y sus paquetes en un entorno aislado (también en Windows)	conda-forge.org/download
R + stylo	Estilometría clásica: Delta, dendrogramas, consenso	Eder, Rybicki y Kestemont (2016), *R Journal* 8(1); paquete en CRAN
Claude Code	El agente: terminal, aplicación de escritorio o extensión de VS Code	code.claude.com/docs
Terminal	Entorno en el que trabaja el agente	Programming Historian, *Introducción a Bash*

No es necesario dominarlas, porque el agente las usa. Sí conviene conocerlas para saber qué pedir y entender qué hace.

Materiales del taller (presentación, prompts, corpus y scripts): github.com/grmarco/taller-agentes-hd

Paso 1: chat y agente

>_ P01 · Chat: pedir el texto

Necesito el texto del Lazarillo de Tormes (1554) en una edición de dominio público para un análisis estilométrico. Descárgamelo y dime cuántas palabras tiene.

El chat devuelve enlaces e instrucciones. Descargar, limpiar y contar queda a cargo del usuario.

>_ P02 · Agente: mismo encargo, con comprobación

Necesito el texto del Lazarillo de Tormes (1554) en una edición de dominio público para un análisis estilométrico. Guarda el original tal cual y prepara aparte una versión limpia, sin cabeceras, notas ni las interpolaciones de la edición de Alcalá. Antes de terminar, comprueba que la versión limpia tiene entre 17 000 y 20 000 palabras y muéstrame la primera y la última frase.

El agente descarga, limpia, cuenta y **comprueba** antes de terminar.

⌨ **En este paso:** `Mayús+Tab`: modos de permiso · `Esc`: interrumpir · el encargo incluye su comprobación

Paso 2: instrucciones permanentes

`CLAUDE.md` o `AGENTS.md`: se lee al empezar cada sesión

P03 · Instrucciones permanentes

```
/init
```

P04 · Completar CLAUDE.md

Añade a CLAUDE.md, en no más de seis líneas: escribe en español idiomático, porque los informes los leen filólogos; ninguna afirmación sin su fuente o sin el resultado que la respalda; no modifiques nunca los textos originales descargados; distingue siempre afinidad estilística de autoría.

En este paso: `/init` lee `PROYECTO.md` y crea `CLAUDE.md` · `/context`: comprobar que se ha cargado

Paso 3: planificar la investigación

P05 · Planificar la investigación

/plan Lee PROYECTO.md y propón un plan de trabajo con subagentes: estado de la cuestión, construcción del corpus (un subagente por candidato), validación del método y análisis. Para cada subagente indica qué modelo usa y por qué (Haiku para descargar y limpiar, Sonnet para buscar y leer bibliografía), cuántos turnos como máximo y cuánto calculas que costará. Incluye cómo comprobaremos que cada parte está bien hecha. No ejecutes nada todavía.

Qué revisar en el plan

- ¿Distingue tareas que piden criterio de tareas mecánicas?
- ¿El coste estimado es razonable?
- ¿Cómo sabremos que cada parte está bien?

Lo que solo puede decidir el investigador: qué candidatos entran, qué ediciones son comparables, qué se considera prueba.

En este paso: `Mayús+Tab` hasta el modo plan · `Ctrl+G`: editar el plan antes de aprobarlo

Paso 4: definir subagentes

Cada subagente tiene su tarea, su modelo y sus límites

P06 · Definir los subagentes

Crea tres subagentes para este proyecto. Un bibliógrafo (con Sonnet y búsqueda en internet) que compruebe que cada estudio existe antes de citarlo y me devuelva un resumen de diez líneas. Un constructor de corpus (con Haiku, el modelo barato) que busque, descargue y limpie los textos en dominio público de un autor y anote de dónde salen. Y un revisor (con Sonnet) que solo pueda leer, no modificar.

En este paso: Haiku para lo mecánico, Sonnet para leer bibliografía · `/usage`: consumo antes y después

Paso 5: orquestar el trabajo en paralelo

P07 · Orquestar el trabajo en paralelo

Ejecuta el plan: lanza a la vez el bibliógrafo y un constructor de corpus por cada candidato con textos en dominio público (Alfonso de Valdés, Juan de Valdés, Diego Hurtado de Mendoza) y otro para los textos de control (Segunda parte de 1555, Juan de Luna 1620, El Crótalon de Villalón). Cuando terminen, resume en una tabla qué tenemos de cada candidato y qué nos falta respecto a los estudios previos.

Qué ocurre

- El bibliógrafo (Sonnet) busca los estudios y comprueba que existen.
- Un constructor de corpus (Haiku) por candidato: descarga, limpia y registra la procedencia.
- Cada uno trabaja con su propia ventana y devuelve un resumen breve.
- El agente principal reúne: qué tenemos y qué falta.

Tarea de días para un becario; aquí, minutos y unos pocos dólares.

En este paso: cada subagente tiene su propia ventana: `/context` de la sesión principal apenas crece

Paso 6: convertir el procedimiento en una *skill*

P08 · Convertir en skill

Convierte en una skill llamada anadir-texto lo que hemos hecho para añadir un texto al corpus: recibe un enlace, un autor y una obra; descarga, limpia, cuenta palabras, enseña el principio y el final y anota la procedencia. Si la edición tiene derechos o el texto tiene menos de 5 000 palabras, que no lo añada y explique por qué.

P09 · Usar la skill

/anadir-texto https://es.wikisource.org/wiki/La_segunda_parte_de_Lazarillo_de_Tormes Anónimo Segunda_parte_1555

En este paso: `/anadir-texto`: la *skill* solo ocupa ventana cuando se usa, a diferencia de `CLAUDE.md`

Paso 7: validar el método y aplicarlo

P10 · Validar el método

Antes de aplicar el método al Lazarillo, valídalo. Con stylo en R, trocea los textos de autor conocido en fragmentos de unas 18 000 palabras y comprueba si Delta (clásica y coseno, de 100 a 500 palabras más frecuentes) agrupa cada fragmento con su autor. Repítelo con la ortografía normalizada. Da el método por bueno solo si acierta al menos el 90 % de los fragmentos; si no llega, explícame por qué y no sigas.

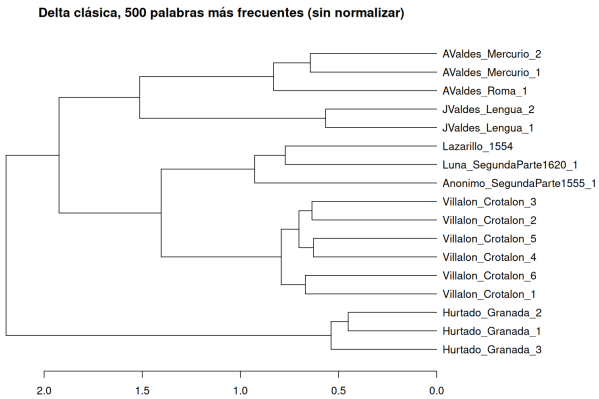
P11 · Aplicar el método al Lazarillo

Aplica al Lazarillo la configuración que mejor ha funcionado: dendrograma, tabla de distancias a cada candidato y árbol de consenso. Escribe un informe que compare con el estado de la cuestión y separe lo que se puede afirmar de lo que no.

La validación es una **condición de parada**: si falla con autores conocidos, no se sigue.

En este paso: `/clear` antes de empezar el análisis · el método se puede comprobar; la interpretación, no

Resultado del caso práctico



- El método distingue a los autores conocidos (14 de 14 fragmentos; también entre obras distintas de Alfonso de Valdés).
- El *Lazarillo* se agrupa con sus continuaciones (1555 y 1620) y después con *El Crótalon*.
- Hurtado queda lejos, pero su texto es una crónica en edición del siglo XIX: ¿autor, género o edición?
- Falta Otálora. **Afinidad, no autoría.**

Delta clásica, 500 palabras más frecuentes. Corpus, scripts e informe: github.com/grmarco/taller-agentes-hd.

Paso 8: proteger los textos descargados

P12 · Hook que protege los textos

Añade un hook, una regla que se cumpla siempre, que impida modificar los textos originales descargados. Después explícame por qué es más fiable que pedírtelo en CLAUDE.md.

Un *hook* se ejecuta siempre. A diferencia de una instrucción, no depende de que el modelo la siga.

En este paso: `Esc Esc` o `/rewind`: deshacer ediciones (no los efectos de los comandos: para eso, git)

Paso 9: revisión independiente

P13 · Revisión independiente

Pide al revisor que examine los resultados y los scripts. Quiero solo los problemas que afecten a la validez de las conclusiones.

Lo que debería encontrar un buen revisor

- Ediciones con ortografía distinta: ¿mide al autor o a la edición?
- Géneros desiguales: diálogo, crónica, novela.
- Falta Otálora, el candidato preferido por los estudios computacionales.
- «Autoría» donde solo hay «afinidad».

En este paso: `/clear`: una sesión nueva revisa sin el sesgo de quien hizo el análisis

Paso 10: control de versiones y ejecución por lotes

P14 · Guardar el trabajo con git

Inicializa un repositorio git si no existe y haz un commit con lo hecho hasta ahora, con un mensaje descriptivo en español.

Git es la red de seguridad: lo que el agente hace con comandos no se puede deshacer con `/rewind`.

P15 · Trabajo por lotes sin conversación

```
for f in datos/limpio/*.txt; do
claude -p "Resume en una línea el género, la
fecha y el registro de $f" –model haiku »
resultados/fichas.txt
done
```

Sin conversación: el mismo encargo sobre todos los textos, con un modelo barato.

En este paso: `claude -p`: sin conversación · probar antes con dos o tres textos · `/usage`: coste total

Reparto del trabajo en el caso práctico

Los agentes

Buscar y comprobar bibliografía, localizar ediciones, descargar, limpiar, registrar procedencias, escribir y ejecutar los scripts, validar el método, redactar el informe.

El investigador

Formular la pregunta, elegir candidatos y ediciones comparables, fijar el criterio de validación, **detectar lo que no tiene sentido filológico** e interpretar.

Iniciar y revisar siguen siendo tareas humanas, y revisar exige conocer la materia.

Alternativas y recomendaciones

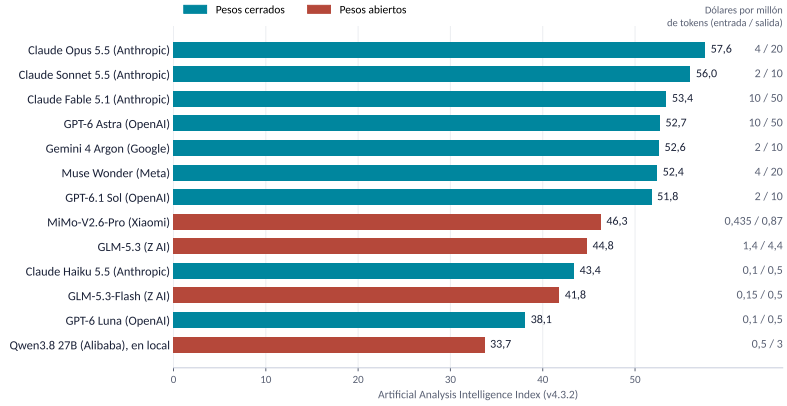
Alternativas a Claude Code

Herramienta	Empresa	Particularidad	Pago
Claude Code	Anthropic	Terminal, escritorio y web	Desde 20 $/mes
Codex	OpenAI	CLI de código abierto (Apache-2.0)	Incluido en ChatGPT
OpenCode	comunidad	Código abierto (MIT); cualquier proveedor y modelos locales	El del modelo que uses
GitHub Copilot	GitHub (Microsoft)	Modo agente; gratis para docentes con GitHub Education	0–39 $/mes
Antigravity	Google	Sustituye a Gemini CLI desde junio de 2026	Planes de Google AI
Cursor	Anysphere	Editor con agentes	0–200 $/mes

Las piezas son las mismas en todas: fichero de instrucciones (`AGENTS.md`), permisos, modo plan, subagentes y *skills*.

Precios en dólares, sin IVA, consultados el 13 y 14-10-2026. Cambian a menudo.

Comparativa de modelos



Datos de artificialanalysis.ai, 13 de octubre de 2026. Mejor variante de razonamiento de cada modelo.

Artificial Analysis, Intelligence Index (consultado el 13-10-2026). Mide tareas de ingeniería, ciencia y oficina, no de humanidades.

Recomendaciones para un primer proyecto

1. Un proyecto pequeño y propio, con datos que conozcas bien.
2. Git desde el primer día.
3. Un fichero de instrucciones breve.
4. Decidir **antes** cómo se comprobará el resultado.
5. Planificar, luego ejecutar.
6. Convertir en *skill* lo que se repite.

Lecturas

- Anthropic, *Best practices for Claude Code*
- Anthropic, *Building effective agents*
- OpenAI, *A practical guide to building agents*
- agents.md
- Documentación: code.claude.com/docs

Gracias

Guillermo Marco Remón
gmarco@lsi.uned.es

Materiales del taller:
github.com/grmarco/taller-agentes-hd

Prompts:
`https://claude.ai/artifact/8E14gReCvbXappyn2BPWnf`



Reserva: estudios estilométricos sobre el *Lazarillo*

Estudio	Método	Conclusión
De la Rosa y Suárez, *Lemir* 20 (2016)	Aprendizaje automático, palabras funcionales	Otálora, seguido de A. de Valdés; sin atribución segura
Rodríguez López-Vázquez, *Lemir* 22 (2018)	Índices léxicos y fraseológicos	Descarta a A. de Valdés y a Hurtado
Madrigal, *Lemir* 26 (2022)	JGAAP y stylo (Delta clásica y coseno)	Otálora; Valdés, Hurtado y Cervantes de Salazar, lejos
Madrigal, *Lemir* 28 (2024)	stylo y paralelos verbales	Otálora
Blasco, *Ínsula* 924 (2023)*	Análisis léxico	Juan de Valdés, el más próximo

*Citado por Martínez Domingo, *eHumanista* 58 (2024); no consultado directamente. Atribuciones filológicas: Navarro Durán (A. de Valdés, 2003), Agulló (Hurtado, 2010), Asensio y Calvo (J. de Valdés).